

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.10.28	完成日期	2024.10.28

第一题

1. 实验题目

检查一个序列是否构成堆

2. 实验目的

给定一个整数序列，请检查该序列是否构成堆。

3. 算法设计

1. 输入容量大小和数组

读取输入的堆容量 n 和存储堆元素的数组 v 。数组 v 按层序存储堆的元素。

2. 处理特殊情况

如果 $n == 1$ ，说明堆中只有一个元素，此时无论是大顶堆还是小顶堆均满足条件，直接输出结果。

3. 利用函数判断堆类型

设计两个函数分别判断是否为大顶堆和小顶堆。

遍历堆中每个非叶子节点，检查其左右子节点是否满足大顶堆或小顶堆的性质。

4. 输出结果

根据两个函数的返回值，输出堆的类型（大顶堆、小顶堆、既非大顶堆也非小顶堆）。

4. 程序运行与测试

1. 判断是否为大顶堆的函数（小顶堆同理）

```
bool is_maxheap(vector<int> v, int n)
```

```

{
    int flag=1;
    int i=0;
    for(i;i<n;i++)
    {
        int left=2*i+1;
        int right=2*i+2;
        if(left<n)
        {
            if(v[left]>v[i])
            {
                flag=0;
                return flag;
            }
        }
        else
            return flag;
        if(right<n)
        {
            if(v[right]>v[i])
            {
                flag=0;
                return flag;
            }
        }
        else
            return flag;
    }
}

```

5. 实验总结与心得

1.大顶堆和小顶堆判定需要分开函数实现

通过分别设计判断大顶堆和小顶堆的函数，可以更加清晰地表达逻辑，避免混淆。

特殊情况如堆既是大顶堆又是小顶堆（如所有元素相等），也可以通过分开实现来处理。

2.注意索引边界问题

判断是否为叶子节点时，if(left < n)和 if(right < n)是关键。因为数组索引从 0 开始，不能写成<=，否则会出现越界访问。

3.程序结构清晰

将不同功能模块化，比如分开设计 is_maxheap 和 is_minheap 函数，主函数只负责调用和输出结果，提高了代码的可读性和复用性。

4.复杂度分析

时间复杂度为 $O(n)$ ，因为需要遍历堆的每个节点。

空间复杂度为 $O(1)$ ，因为算法只需常量额外空间。

第二题

1. 实验题目

奖学金

2. 实验目的

某小学最近得到了一笔赞助，打算拿出其中一部分为学习成绩优秀的前 5 名学生发奖学金。期末，每个学生都有 3 门课的成绩:语 文、数学、英语。先按总分从高到低排序，如果两个同学总分相同，再按语文成绩从高到低排序，如果两个同学总分和语文成绩都 相同，那么规定学号小的同学排在前面，这样，每个学生的排序是唯一确定的。 任务:先根据输入的 3 门课的成绩计算总分，然后按上述规则排序，最后按排名顺序输出前 5 名学生的学号和总分。注意，在前 5 名 同学中，每个人的奖学金都不相同，因此，你必须严格按上述规则排序。例如，在某个正确答案中，如果前两行的输出数据(每行 输出两个数:学号、总分)是:

7 279

5 279

这两行数据的含义是:总分最高的两个同学的学号依次是 7 号、5 号。这两名同学的总分都是 279(总分等于输入的语文、数学、英 语三科成绩之和)，但学号为 7 的学生语文成绩更高一些。如果你的前两名的输出数据是:

5 279

7 279

则按输出错误处理，不能得分。

3. 算法设计

1.使用类（class）来存储每个学生的基本信息

定义一个 Grade 类来存储每个学生的 id、chinese_score、math_score、english_score 和 sum。其中 sum 是学生的总分，需要根据每个学生的各科成绩动态计算得到。

2.输入学生的 id 和成绩

通过循环读取每个学生的 id 及其 chinese_score、math_score、english_score，并计算其总分。

3.使用 sort 函数进行自定义类型排序

利用 C++的 sort 函数对学生数据进行排序。在排序时，需要根据学生的总分进行排序，如果总分相同，则根据语文成绩排序；如果语文成绩也相同，则根据 id 排序。

4.书写自定义类型的排序的重载函数

通过重载 operator()函数来自定义排序规则，使得可以通过 sort 函数进行排序。

4. 程序运行与测试

1. 学生成绩的 class

```
class Grade
{
public:
    int id;
    int chinese;
    int math;
    int english;
    int sum;
};
```

2. 排序的重载函数

```
class my_compare
{
public:
    bool operator()(const Grade& g1,const Grade& g2)
    {
        if(g1.sum==g2.sum)
        {
            if(g1.chinese==g2.chinese)
                return g1.id<g2.id;//成绩相同比较 id
            return g1.chinese>g2.chinese;
        }
        return g1.sum>g2.sum;
    }
};
```

5. 实验总结与心得

1.实验题不算难，和大一学习 C 语言做的 struct 相关题型相似

这道题主要是将结构体升级为类，并运用 C++的特性（如函数重载和 STL 容器）进行排序，整体思路与 struct 排序题目相似。

2.学会利用函数重载，使用 sort 对自定义类型进行排序

通过重载 operator()，可以让自定义类型与 STL 中的 sort 函数配合使用，这种方法非常灵活。在此实验中，利用函数重载实现了对学生成绩的多层次排序，使得排序逻辑更加清晰。

3.函数与类的设计

通过定义 Grade 类和 my_compare 类，成功将问题分解为几个小问题（输入、排序、输出）。这不仅提高了程序的可读性，也增强了代码的模块化，易于维护和扩展。

4.时间复杂度

排序的时间复杂度为 $O(n \log n)$ ，其中 n 是学生的数量。由于 sort 函数采用了快速排序，因此排序的效率较高。

5.扩展性

如果以后需要对更多的字段进行排序，或者改变排序规则，可以很方便地修改 `my_compare` 类中的逻辑，而不需要修改主程序中的其他部分。

第三题

1. 实验题目

heap

2. 实验目的

实现一个小根堆用以做优先队列。

给定如下数据类型的定义：

```
class array {  
  
private:  
  
    int elem[MAXN];  
  
public:  
  
    int &operator[](int i) { return elem[i]; }  
  
};  
  
class heap {  
  
private:  
  
    int n;  
  
    array h;  
  
public:  
  
    void clear() { n = 0; }  
  
    int top() { return h[1]; }  
  
    int size() { return n; }  
  
    void push(int);  
  
    void pop();  
  
};
```

3. 算法设计

1.push 函数（插入操作）

目标： 将一个新元素插入到最大堆中。

步骤：

- a. 将新元素添加到堆的最后一个位置。
- b. 从新插入的元素开始，沿着堆从下往上调整，直到堆满足最大堆的性质（父节点大于子节点）。

核心思想： 最大堆的插入过程与堆的初始化过程相似，都是向堆中添加一个新元素，然后通过“向上调整”保证堆的性质。

2.pop 函数（删除操作）

目标： 删除最大堆中的堆顶元素（即最大值）。

步骤：

- a. 将堆的最后一个节点与堆顶元素交换。
- b. 删除堆顶元素（即将最后一个元素的值移到根节点并删除）。
- c. 从堆顶开始，沿着堆从上往下调整，直到堆满足最大堆的性质。

核心思想： 最大堆的删除操作通过交换堆顶元素与最后一个元素来删除堆顶，然后使用“向下调整”重新调整堆，使其恢复最大堆的结构。

4. 程序运行与测试

1.push 函数//最大堆的插入节点的思想就是先在堆的最后添加一个节点，然后沿着堆树上升。跟最大堆的初始化过程大致相同。

```
void heap::push(int x)
{
    if(n==MAXN)
        return;//overflow
    n++;
    h[n]=x;

    int dad;
    int son=n;
    while(son>1)//向上调整 从最后一个结点开始
    {
        dad=son/2;
        if(h[son]<h[dad])
        {
            swap(h[son],h[dad]);
            son=dad;
        }
        else
            break;
    }
```

```

    }
}

```

2.pop 函数

//最大堆堆顶节点删除思想如下：将堆树的最后的节点提到根结点，然后删除最大值，然后再把新的根节点放到合适的位置。

```

void heap::pop() {
    if(n==0)
        return;//underflow

    swap(h[1],h[n]);//交换第一个与最后一个结点

    n--;//删除最后一个结点

    int dad=1;
    int son=2*dad;
    while(son<=n)//向下调整 从第一个结点开始
    {
        if(son+1<=n && h[son]>h[son+1])//若左右孩子都存在,且右孩子更小
        {
            son++;//将孩子更新为右孩子
        }
        if(h[son]<h[dad])
        {
            swap(h[son],h[dad]);

            dad=son;
            son=2*dad;//下沉后子树改变 继续下沉
        }
        else
            break;
    }
}
}

```

5. 实验总结与心得

1.本题考察了堆的插入与删除操作，尤其是对堆的向上调整和向下调整的实现。
 插入操作使用的是“向上调整”策略，确保每个新插入的元素都能正确地插入到堆中。
 删除操作则采用“向下调整”策略，确保堆顶元素被移除后，堆仍然满足最大堆的性质。

2.利用循环操作进行堆调整，避免了递归带来的额外开销。

向上调整和向下调整的操作都使用了循环，确保操作的简洁性和高效性，特别是在处理较大的堆时，这种做法避免了递归带来的栈空间消耗。

3.注意堆数组中元素的索引管理。

在堆数组中， $h[1]$ 是堆顶元素，左子节点为 $2 * i$ ，右子节点为 $2 * i + 1$ 。这些索引的管理需要特别注意，尤其在进行插入和删除操作时，son 和 dad 的更新要非常清晰。

4.类型定义与函数重载。

在实现堆操作时，确保正确使用了堆的数组表示和相应的操作逻辑。通过函数的设计，我们成功地对堆的结构进行了有效的操作与管理。

5.实验中的优化与实践：

在调整过程中，使用了 while 循环代替递归，不仅能减少函数调用栈的消耗，还能提高代码的执行效率。通过这种实验，可以更好地理解堆数据结构的运作方式，并掌握如何使用堆进行高效的插入与删除操作。